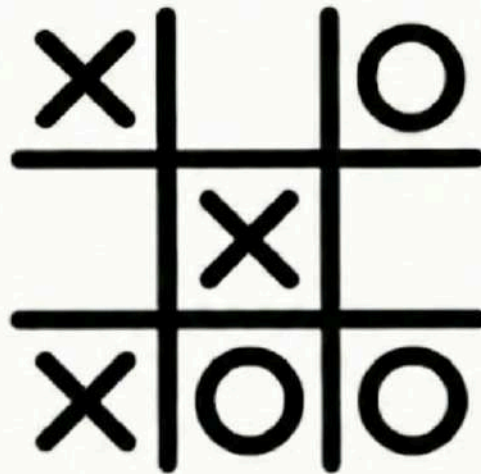


TIC TAC TOE

A PROJECT REPORT



SUBMITTED BY: KRISHNA PALIWAL

REGISTRATION NO.: 25BAI11317

DEPARTMENT OF COMPUTER SCIENCE

Introduction

The game of Tic-Tac-Toe is an example of a simple task which can be programmed in Java by a beginner.

Standard Loops, Methods, Conditional statements and Arrays are the main features of Java language that are illustrated in the project to have a working game.

Problem Statement

Design and develop a functional Java program that allows two human players to play Tic-Tac-Toe on a 3×3 grid. The system verifies the moves, updates the board, detects the winner, and determines if the match ends in a draw.

Project Overview

Tic-Tac-Toe is a classic game played on a 3×3 grid between two players. Player X and Player O take turns.

We prompt the users to select the grid positions on the 3×3 board by entering numbers between 0 and 8. The mark is placed in the selected position.

The board is updated after every input, and the program verifies a player's win.

Functional Requirements

FR1 – Player Input Module

- Accepts moves (0–8) from Player X and Player O.
- Handles sequential turn-taking between players.

FR2 – Board Management Module

- Stores and updates the 3×3 grid using arrays.
- Displays board after each turn.

FR3 – Winner/Draw Evaluation Module

- Checks 8 winning combinations (3 rows, 3 columns, 2 diagonals)
- Declares winner or draw

Non-Functional Requirements

1. Usability:

The game must be easy to understand, with clear prompts.

2. Reliability:

Should correctly identify all winning patterns and avoid invalid moves.

3. Performance:

The program should respond instantly as it is lightweight.

4. Maintainability:

Code is modular with separate functions for board display and winner checking.

5. Error-Handling:

Prevents invalid inputs and ensures smooth gameplay.

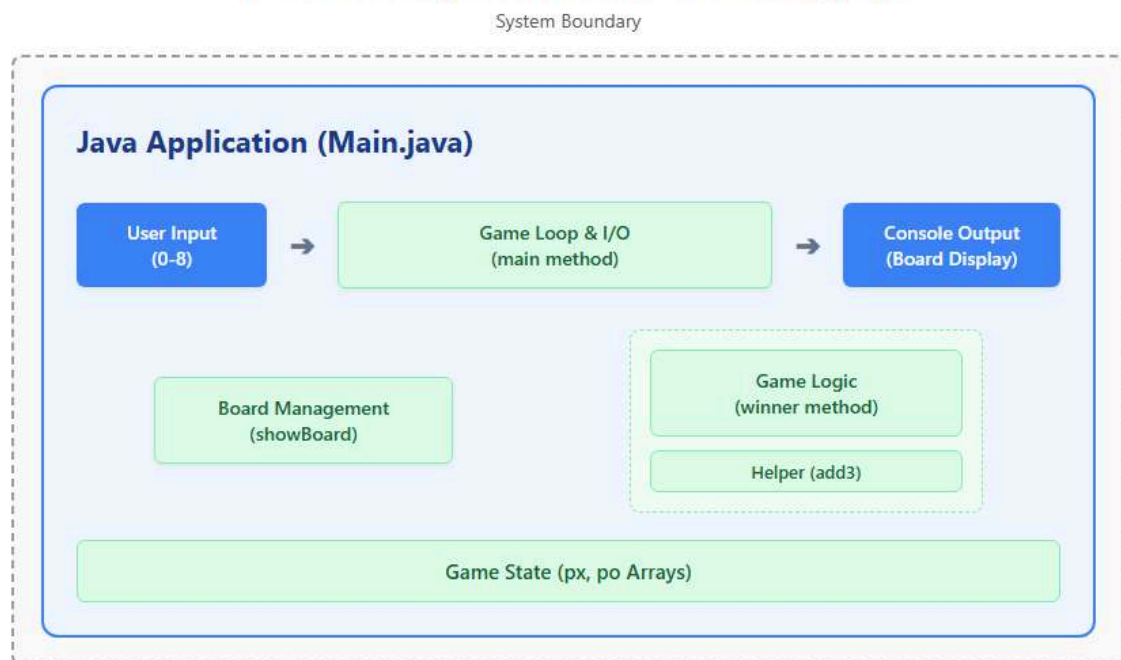
System Architecture

Architecture Description

The system follows a simple linear structure:

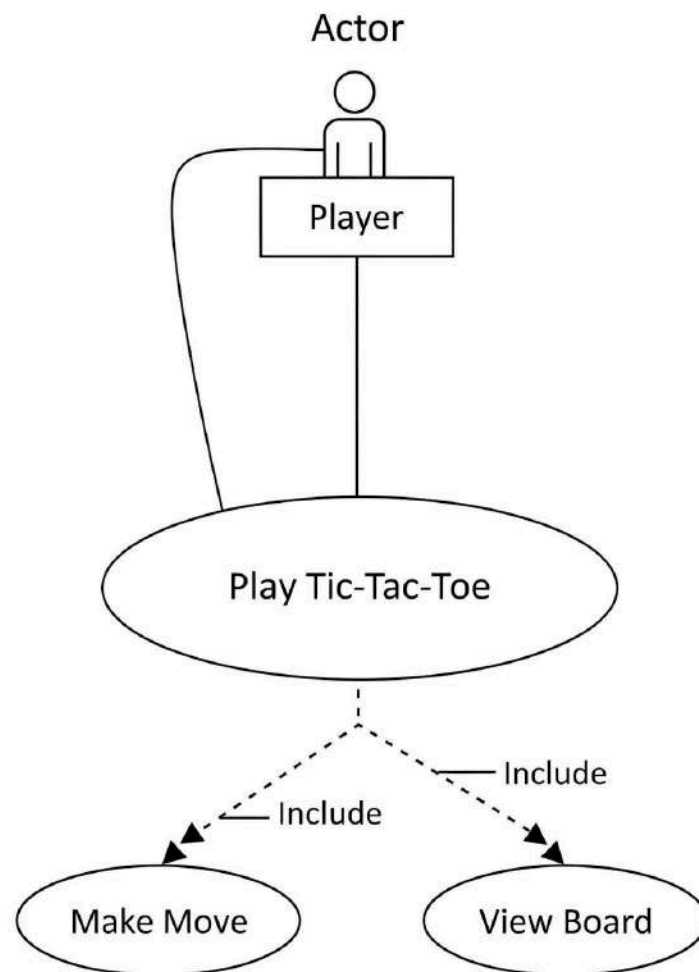
Player Input → Board Update → Check Winner → Check Draw → Repeat

Tic-Tac-Toe System Architecture Diagram

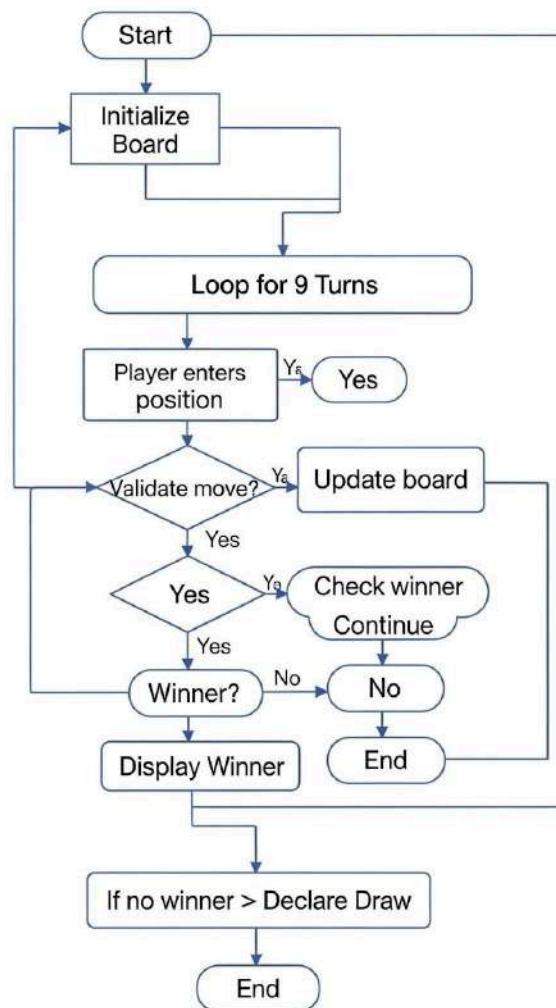


Design Diagram

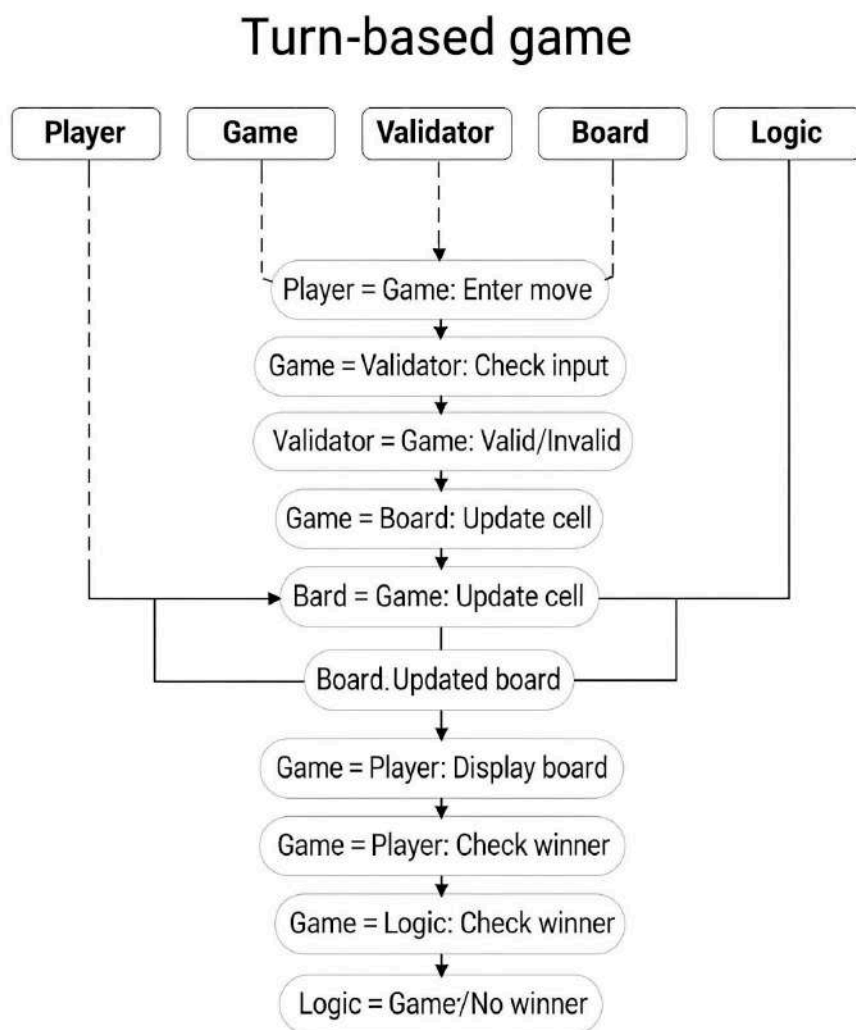
1. Use Case Diagram



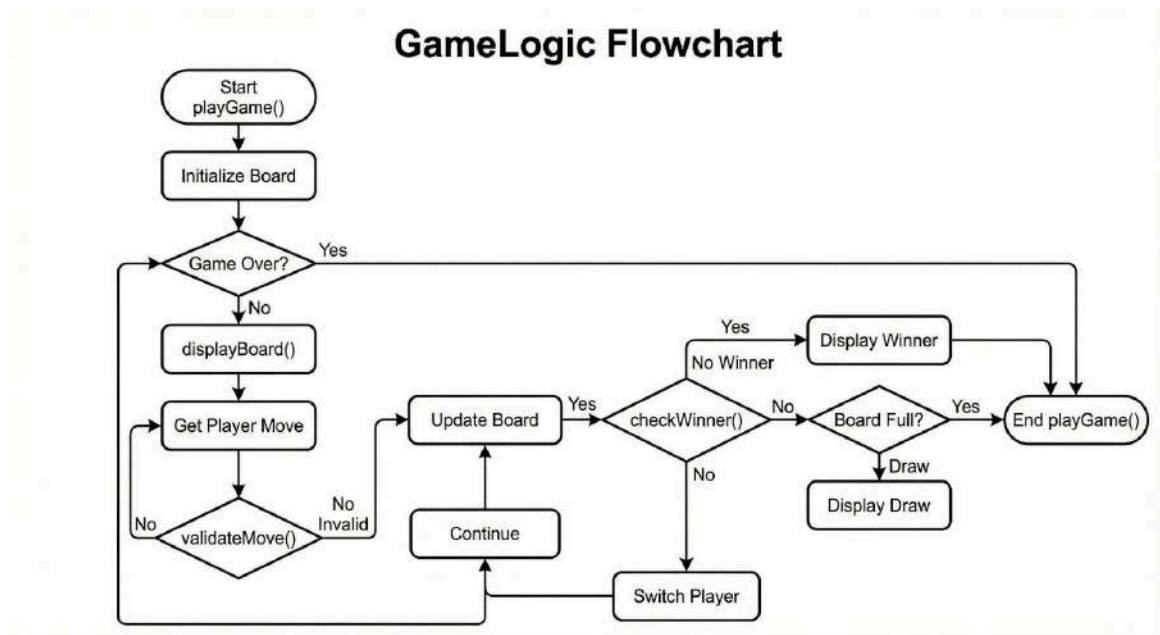
2. Work Flow Diagram



3. Sequence Diagram



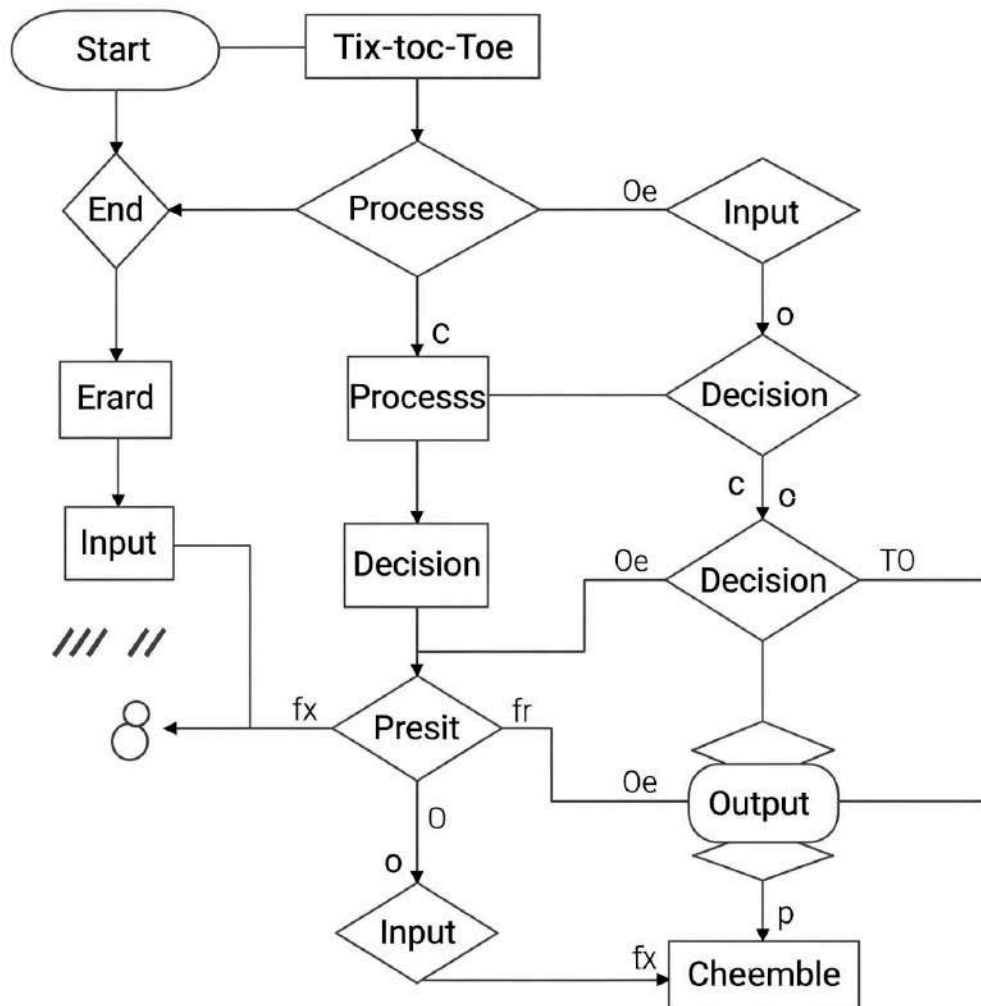
4. Class / Component Diagram



Design Decisions & Rationale

Design Choices

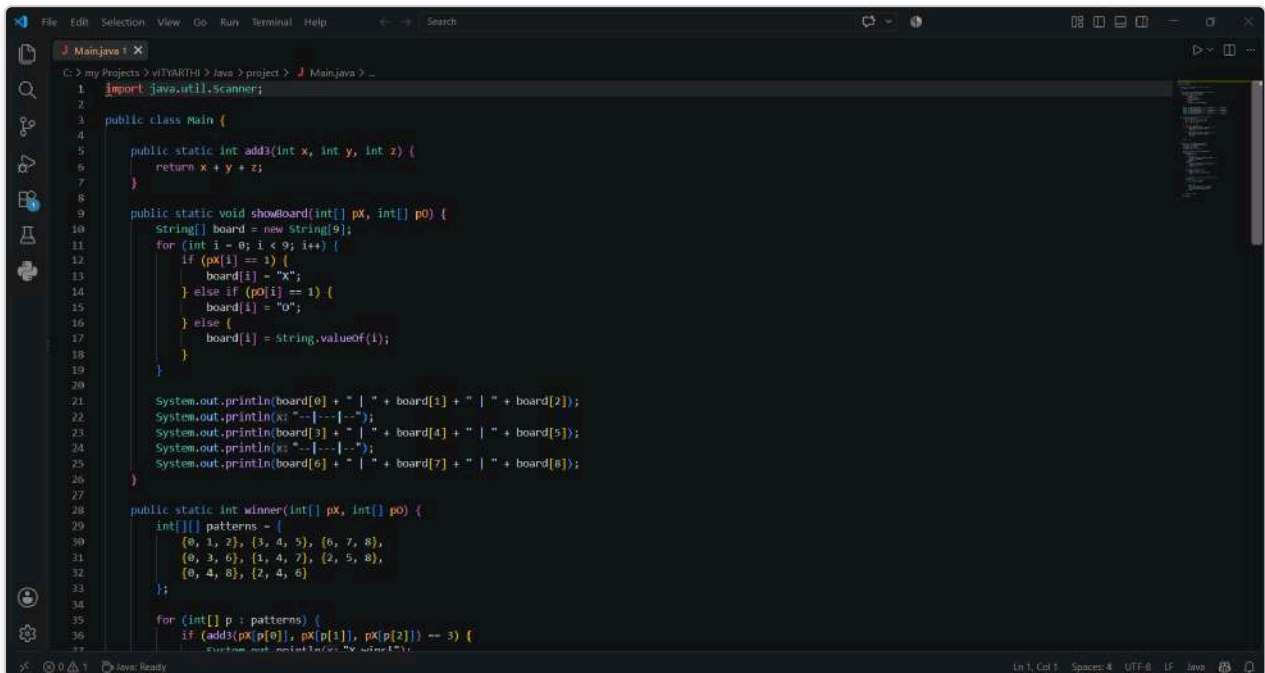
Design Choices



Implementation Details

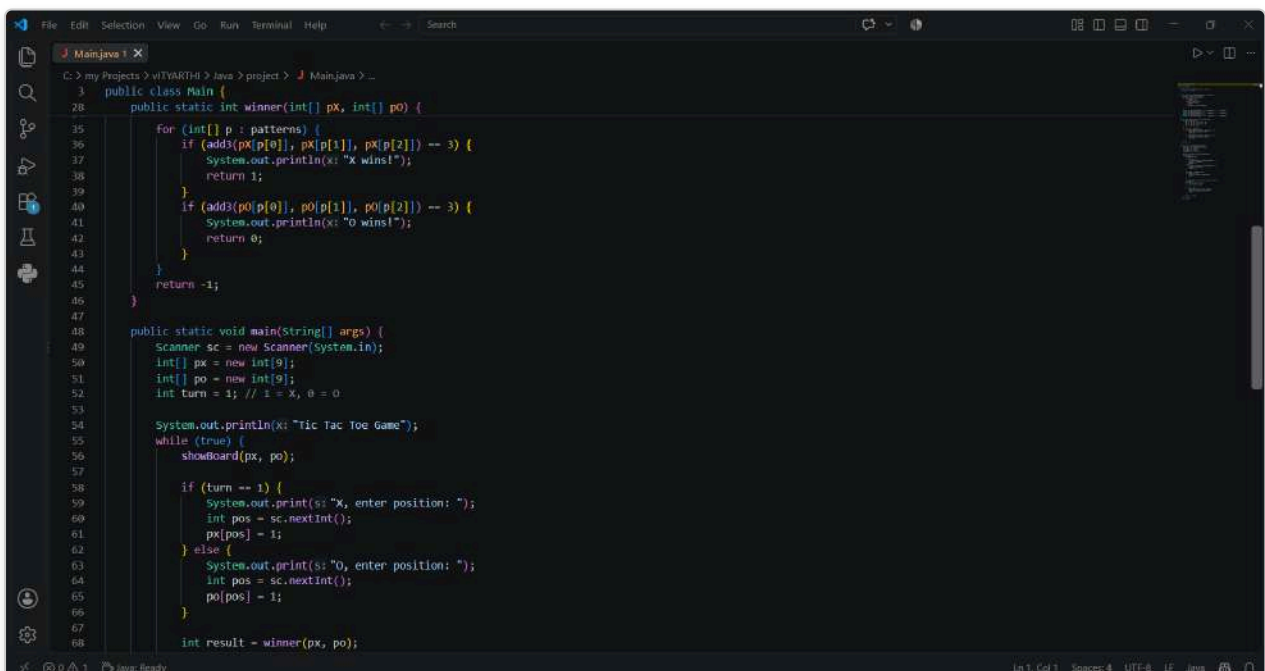
1. **Arrays:** The 3×3 board and player states are stored and managed using 1D integer and String arrays of 9 elements.
2. **Functions/Methods:** Employed to show the board (showBoard), evaluate cells (add3), and find the winners (winner).
3. **Conditionals:** Ensure that moves are processed, identify winners, and take care of games that end in a draw.
4. **Loops:** There can be a maximum of 9 turns, thus the game may loop 9 times. Someone may also win before that.
5. After the game is initialized, the board is displayed. The players decide which numbered position (0 to 8) is going to be their move.
6. The program updates the board after every turn and checks all possible winning combinations.
7. If none of the combinations satisfy the winning condition and the board is full, the result is a draw.

Screen Shots and Results



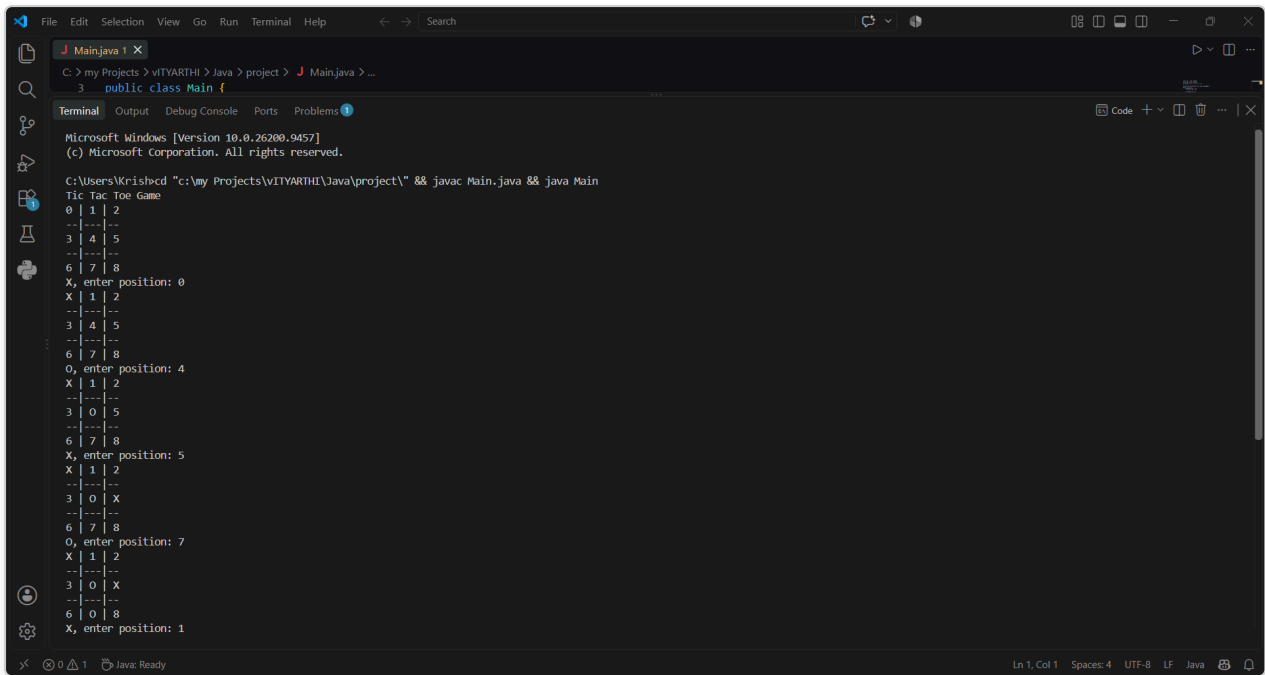
```
1  import java.util.Scanner;
2
3  public class Main {
4
5      public static int add3(int x, int y, int z) {
6          return x + y + z;
7      }
8
9      public static void showBoard(int[] px, int[] po) {
10         String[] board = new String[9];
11         for (int i = 0; i < 9; i++) {
12             if (px[i] == 1) {
13                 board[i] = "X";
14             } else if (po[i] == 1) {
15                 board[i] = "O";
16             } else {
17                 board[i] = String.valueOf(i);
18             }
19         }
20
21         System.out.println(board[0] + " | " + board[1] + " | " + board[2]);
22         System.out.println("---|---|---");
23         System.out.println(board[3] + " | " + board[4] + " | " + board[5]);
24         System.out.println("---|---|---");
25         System.out.println(board[6] + " | " + board[7] + " | " + board[8]);
26     }
27
28     public static int winner(int[] px, int[] po) {
29         int[][] patterns = {
30             {0, 1, 2}, {3, 4, 5}, {6, 7, 8},
31             {0, 3, 6}, {1, 4, 7}, {2, 5, 8},
32             {0, 4, 8}, {2, 4, 6}
33         };
34
35         for (int[] p : patterns) {
36             if (add3(px[p[0]], px[p[1]], px[p[2]]) == 3) {
37                 return 1;
38             }
39             if (add3(po[p[0]], po[p[1]], po[p[2]]) == 3) {
40                 return 0;
41             }
42         }
43         return -1;
44     }
45
46     public static void main(String[] args) {
47         Scanner sc = new Scanner(System.in);
48         int[] px = new int[9];
49         int[] po = new int[9];
50         int turn = 1; // 1 = X, 0 = O
51
52         System.out.println("Tic Tac Toe Game");
53         while (true) {
54             showBoard(px, po);
55
56             if (turn == 1) {
57                 System.out.print("X, enter position: ");
58                 int pos = sc.nextInt();
59                 px[pos] = 1;
60             } else {
61                 System.out.print("O, enter position: ");
62                 int pos = sc.nextInt();
63                 po[pos] = 1;
64             }
65
66             int result = winner(px, po);
67             if (result != -1) {
68                 System.out.println("Player " + (result == 1 ? "X" : "O") + " wins!");
69                 break;
70             }
71             turn = 1 - turn;
72         }
73     }
74 }
```

Figure 1: VS Code Environment - Main.java source code (Part 1).



```
28     public static int winner(int[] px, int[] po) {
29         int[][] patterns = {
30             {0, 1, 2}, {3, 4, 5}, {6, 7, 8},
31             {0, 3, 6}, {1, 4, 7}, {2, 5, 8},
32             {0, 4, 8}, {2, 4, 6}
33         };
34
35         for (int[] p : patterns) {
36             if (add3(px[p[0]], px[p[1]], px[p[2]]) == 3) {
37                 System.out.println("X wins!");
38                 return 1;
39             }
40             if (add3(po[p[0]], po[p[1]], po[p[2]]) == 3) {
41                 System.out.println("O wins!");
42                 return 0;
43             }
44         }
45         return -1;
46     }
47
48     public static void main(String[] args) {
49         Scanner sc = new Scanner(System.in);
50         int[] px = new int[9];
51         int[] po = new int[9];
52         int turn = 1; // 1 = X, 0 = O
53
54         System.out.println("Tic Tac Toe Game");
55         while (true) {
56             showBoard(px, po);
57
58             if (turn == 1) {
59                 System.out.print("X, enter position: ");
60                 int pos = sc.nextInt();
61                 px[pos] = 1;
62             } else {
63                 System.out.print("O, enter position: ");
64                 int pos = sc.nextInt();
65                 po[pos] = 1;
66             }
67
68             int result = winner(px, po);
69             if (result != -1) {
70                 System.out.println("Player " + (result == 1 ? "X" : "O") + " wins!");
71                 break;
72             }
73             turn = 1 - turn;
74         }
75     }
76 }
```

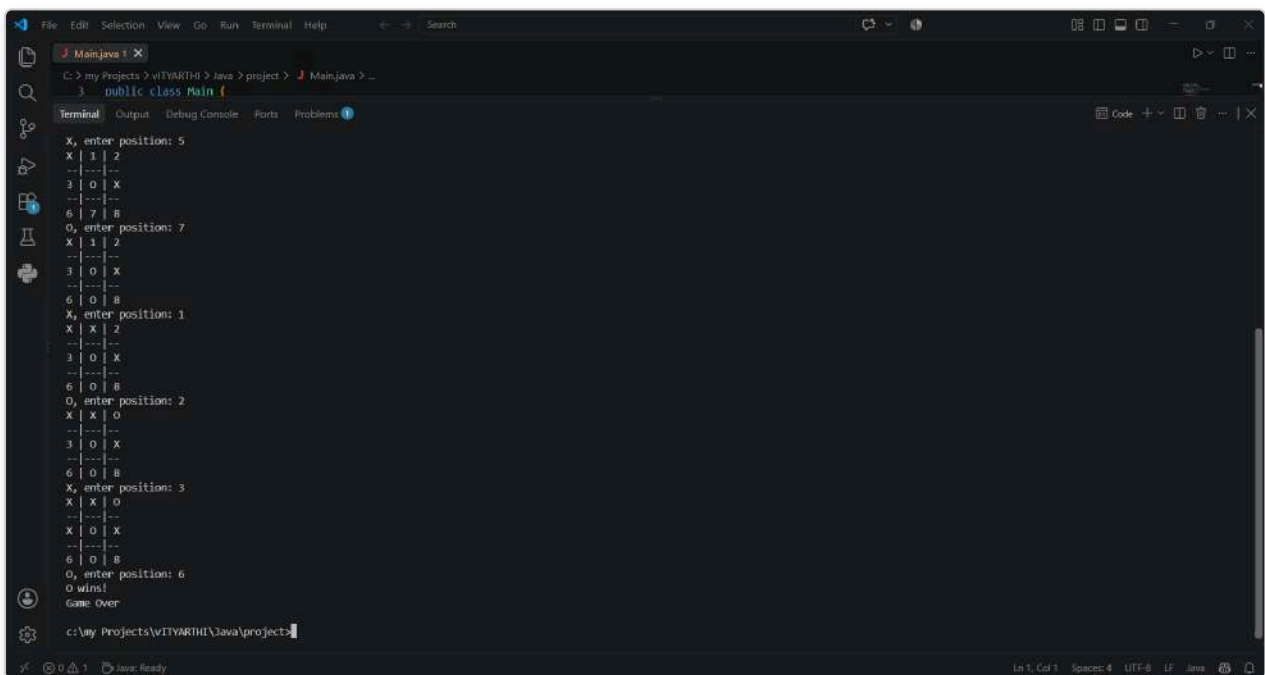
Figure 2: VS Code Environment - Main.java source code (Part 2).



```
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Kishor>cd "c:\my Projects\VTIYARTHI\Java\project\" && javac Main.java && java Main
Tic Tac Toe Game
0 | 1 | 2
--|---|--
3 | 4 | 5
--|---|--
6 | 7 | 8
X, enter position: 0
X | 1 | 2
--|---|--
3 | 4 | 5
--|---|--
6 | 7 | 8
O, enter position: 4
X | 1 | 2
--|---|--
3 | 0 | 5
--|---|--
6 | 7 | 8
X, enter position: 5
X | 1 | 2
--|---|--
3 | 0 | X
--|---|--
6 | 7 | 8
O, enter position: 7
X | 1 | 2
--|---|--
3 | 0 | X
--|---|--
6 | 0 | 8
X, enter position: 1
```

Figure 3: Program Execution and Move Input in Terminal.



```
X, enter position: 5
X | 1 | 2
--|---|--
3 | 0 | X
--|---|--
6 | 7 | 8
O, enter position: 7
X | 1 | 2
--|---|--
3 | 0 | X
--|---|--
6 | 0 | 8
X, enter position: 1
X | X | 2
--|---|--
3 | 0 | X
--|---|--
6 | 0 | 8
O, enter position: 2
X | X | 0
--|---|--
3 | 0 | X
--|---|--
6 | 0 | 8
X, enter position: 3
X | X | 0
--|---|--
X | 0 | X
--|---|--
6 | 0 | 8
O, enter position: 6
O wins!
Game Over

c:\my Projects\VTIYARTHI\Java\project>
```

Figure 4: Match Outcome and Winner Declaration in Terminal.

Testing And Approach

1. Test valid moves (positions 0–8)
2. Test sequential player switching (X then O)
3. Test winning rows/columns/diagonals
4. Test draw condition
5. Test early termination when winner found

Challenges Faced

1. Checking all winning combinations correctly
2. Translating Python dynamic structures into clean Java arrays
3. Making the board display clean and readable in console

Learnings & Key Takeaways

1. Understanding of Java methods and modular programming
2. Improved debugging skills and CLI execution

3. Realizing how simple logic builds complete systems
4. Experience with conditional branching and loops in Java

Future Enhancements

1. Add computer AI opponent (Minimax)
2. Add scoring system
3. Add GUI using Java Swing or JavaFX
4. Add multiplayer over network

Reference

1. Vityarthi
2. Oracle Java Documentation
3. GitHub Open Source
4. YouTube Tutorials

Logic to Win

The program checks eight different sequences of cells that might bring victory:

- Three rows (0-1-2, 3-4-5, 6-7-8)
- Three columns (0-3-6, 1-4-7, 2-5-8)
- Two diagonals (0-4-8, 2-4-6)

- i.e., any three cells which contain the same symbol, X or O, are sufficient to declare that player the winner.

Conclusion

This Tic Tac Toe project is a demonstration of how simple concepts can be used to create a working game which solves the problem.

It is a good pool of exercises for programming novices learning Java and may be developed further by adding features like AI player, GUI, scoreboard, etc.